

1. Prove Fermat's little theorem from group-theoretic first principles, then explain how it motivates the Miller-Rabin probabilistic primality test. Apply the test (base $a=2$) to $n=561$ and explain why 561 is a Carmichael number, that fools the simple Fermat test.

⇒ Fermat's Little theorem:

If p is prime number and a is integer such that $p \nmid a$ then;

$$a^{p-1} \equiv 1 \pmod{p}$$

Proof from group-theoretic first principle:

Let $G = (\mathbb{Z}/p\mathbb{Z})^\times$ be the multiplicative group of nonzero residue classes modulo p .

Since, p is prime, every nonzero residue has multiplicative inverse.

$$|G| = p-1$$

By Lagrange's theorem;

$$\text{ord}(a) \mid p-1$$

therefore, $a^{p-1} = e$

for modular arithmetic $e = 1$;

$$a^{p-1} \equiv 1 \pmod{p}$$

Motivation for Miller-Rabin test

Fermat's little theorem provides:

$$a^{p-1} \equiv 1 \pmod{p}$$

still some composite numbers satisfy the condition for certain bases, does not guarantee the n is prime.

The miller test improves it by examining the structure of exponentiation.

For an odd integer $n > 2$,

$$n-1 = 2^s d, \text{ where } d \text{ is odd.}$$

For chosen base a ,

$$x = a^d \pmod{n}$$

It passes the test with base a if:

- ① $x \equiv 1 \pmod{n}$
- ② $x \equiv n-1 \pmod{n}$
- ③ Repeatedly squaring x , $n-1$ obtained before 1.

For $n = 561$, base $a = 2$

$$n-1 = 561 - 1 = 560$$

$$\therefore 560 = 2^4 \times 35$$

$$s = 4, d = 35$$

now, $x = a^s \pmod n$

$$2^{35} \pmod{561} = 263$$

$$263^2 \pmod{561} = 166$$

$$166^2 \pmod{561} = 67$$

$$67^2 \pmod{561} = 1$$

$$263 \rightarrow 166 \rightarrow 67 \rightarrow 1$$

None of them is 560 and initial value is 1, Failed the test, hence composite not prime.

How 561 fools Fermat test:

$561 = 3 \times 11 \times 17$, hence, ^{clearly} composite number.

But Fermat's test says:

$$a^{n-1} \equiv 1 \pmod n$$

for base $a=2$;

$$2^{560} \bmod 561 = 1$$

$$\text{also } \gcd(2, 561) = 1$$

so, Fermat's theorem says it's prime, but clearly it's composite.

Miller-Rabin avoids this by checking squaring values.

2. Prove that, $\mathbb{Z}$ under addition and multiplication modulo n , is a field if and only if n is prime. Illustrate your proof by constructing the multiplicative structure of $\mathbb{Z}_{11}$ with $\mathbb{Z}_{12}$.

$\Rightarrow$ Let,
 $\mathbb{Z}_n = \{0, 1, 2, \dots, n-1\}$
with addition and multiplication performed modulo n .

Proving two ways;

① If n is prime then $\mathbb{Z}_n$ is field.

Let $n=p$ where p is prime.

For every nonzero $a \in \mathbb{Z}_p$,

$$1 \leq a \leq p-1$$

therefore,

$$\gcd(a, p) = 1$$

by Bezout's identity,

$$ab + pk = 1$$

taking modulo p ,

$$ab \equiv 1 \pmod{p}$$

hence, a has ~~non~~ multiplicative inverse b . $\mathbb{Z}_p$ is field.

② If n is composite,

$$n = ab$$

$$1 < a < n, 1 < b < n$$

therefore,

$$ab \equiv 0 \pmod{n}$$

but $a \neq 0, b \neq 0$; they are zero-divisors,

But a field cannot have zero-divisors; hence $\mathbb{Z}_n$ is not field.

For $\mathbb{Z}_{11}$:

$$\mathbb{Z}_{11} = \{0, 1, 2, 3, 4, \dots, 10\}$$

Here even nonzero has a multiplicative inverse;

$$2 \times 6 = 12 \equiv 1 \pmod{11}, 2^{-1} = 6$$

$$3 \times 4 = 12 \equiv 1 \pmod{11}, 3^{-1} = 4$$

Hence, Z_{11} is field, every non-zero has multiplicative inverse.

For Z_{12} :

$$12 = 3 \times 4$$

$$3 \times 4 \equiv 0 \pmod{12}$$

Here 3 and 4 are zero divisors.

3 has no multiplicative inverse, there is no x satisfying:

$$3x \equiv 1 \pmod{12}$$

Hence, Z_{12} is not field.

Question 3: Construct $GF(2^3)$ using the irreducible polynomial $x^3 + x + 1$ over $GF(2)$. Build the multiplication table for a generator element α and determine whether the multiplicative group $GF(2^3)^*$ is cyclic.

$\Rightarrow$ A degree-3 irreducible polynomial over $GF(2)$ produces a field with $2^3 = 8$ elements.

$$GF(2^3) = GF(8)$$

Given the irreducible polynomial:

$$p(x) = x^3 + x + 1$$

Let α be the root of $p(x)$:

$$\alpha^3 + \alpha + 1 = 0$$

For arithmetic over $GF(2)$, $1 = -1$;

$$\alpha^3 = \alpha + 1$$

therefore

$$GF(8) = \{0, 1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$$

Using $\alpha^3 = \alpha + 1$, powers of α :

$$\alpha^4 = \alpha(\alpha^3) = \alpha^2 + \alpha$$

$$\alpha^5 = \alpha(\alpha^4) = \alpha^3 + \alpha^2 = \alpha + 1 + \alpha^2 = \alpha^2 + \alpha + 1$$

$$\alpha^6 = \alpha(\alpha^5) = \alpha^3 + \alpha^2 + \alpha = \alpha + 1 + \alpha^2 + \alpha = \alpha^2 + 1$$

$$\alpha^7 = \alpha(\alpha^6) = \alpha^3 + \alpha = \alpha + 1 + \alpha = 1$$

thus, $\alpha^7 = 1$

For the multiplication table,

$$\text{since, } \alpha^7 = 1$$

$$\alpha^i \alpha^j = \alpha^{i+j \pmod{7}}$$

x	1	α	α^2	α^3	α^4	α^5	α^6
1	1	α	α^2	α^3	α^4	α^5	α^6
α	α	α^2	α^3	α^4	α^5	α^6	1
α^2	α^2	α^3	α^4	α^5	α^6	1	α
α^3	α^3	α^4	α^5	α^6	1	α	α^2
α^4	α^4	α^5	α^6	1	α	α^2	α^3
α^5	α^5	α^6	1	α	α^2	α^3	α^4
α^6	α^6	1	α	α^2	α^3	α^4	α^5

$\mathbb{GF}(8)^*$ is cyclic:

The multiplicative group of non-zero elements is:

$$\mathbb{GF}(8)^* = \{1, \alpha, \alpha^2, \alpha^3, \alpha^4, \alpha^5, \alpha^6\}$$

it contains $8 - 1 = 7$ elements.

Since, $\alpha^7 = 1$ and no smaller power of α equals 1, the order of α is:

$$\text{ord}(\alpha) = 7$$

thus, α generates all non-zero 7 elements.

So, $\mathbb{GF}(8)^*$ is cyclic.

Question 4: Derive the time complexity of modular exponentiation via square-and-multiply. Compute $7^{560} \bmod 561$ step by step and interpret the result in light of 561 being a Carmichael number.

$\Rightarrow$

Time complexity: To compute $a^e \bmod n$, directly multiplying a by itself e times would require $O(e)$ multiplications.

The square-and-multiply method represents the exponent e in binary form and repeatedly square the result.

For example, $560 = \text{512} + 32 + 16$

$$560 = 512 + 32 + 16$$

$$\begin{aligned} \text{So, } 7^{560} &= 7^{512+32+16} \\ &= 7^{512} \times 7^{32} \times 7^{16} \end{aligned}$$

The number of squaring operations is proportional to the number of bits in e , which leads to

$$\lfloor \log_2 e \rfloor + 1$$

Thus, time complexity $O(\log e)$.

Computing $7^{560} \bmod 561$:

Step 1: $560 = (10000110000)_2$

$$560 = 512 + 32 + 16$$

$$\text{Therefore } 7^{560} = 7^{512} \times 7^{32} \times 7^{16}$$

Step 2: Repeatedly squaring:

$$7^2 = 49$$

$$7^4 = 157 \pmod{516}$$

$$7^8 = (157)^2 = 526 \pmod{561}$$

$$7^{16} = (526)^2 = 103 \pmod{561}$$

$$7^{32} = (103)^2 = 511 \pmod{561}$$

$$7^{64} = 67^2 = 256 \pmod{561}$$

$$7^{128} = 496^2 = 529 \pmod{561}$$

$$7^{256} = 529^2 = 463 \pmod{561}$$

$$7^{512} = 463^2 = 298 \pmod{561}$$

$$7^{64} = 256 \pmod{561}$$

$$7^{128} = 460 \pmod{561}$$

$$7^{256} = 103 \pmod{561}$$

$$7^{512} = 511 \pmod{561}$$

Step 3: substituting the calculated values

$$7^{560} \equiv 292 \times 67 \times 103 \pmod{561}$$

~~7^{560}~~

$$292 \times 67 \equiv 334 \pmod{561}$$

$$334 \times 103 \equiv 1 \pmod{561}$$

Step 3: substituting the calculated values:

$$7^{560} \equiv 511 \times 511 \times 103 \pmod{561}$$

$$511 \times 511 \equiv 256 \pmod{561}$$

$$256 \times 103 \equiv 1 \pmod{561}$$

$$\text{Hence, } 7^{560} \equiv 1 \pmod{561}$$

561 as a Carmichael Number:

We know that, $561 = 3 \times 11 \times 17$, hence it's clearly composite.

$$\text{However, } 7^{560} \equiv 1 \pmod{561}$$

So, it satisfies the Fermat's primality condition for base 7.

$$a^{n-1} \equiv 1 \pmod{n}$$

So, Fermat test will incorrectly regard 561 as possible prime. But it's clearly composite, hence it's a Carmichael number.

Question 5: Compare deterministic (AKS) and probabilistic (Miller-Rabin) primality testing in terms of asymptotic complexity, error probability and practical relevance to RSA key generation at 2048-bit and 4096-bit key size.

⇒ AKS (Agarwal-Kayal-Saxena) algorithm is a deterministic primality test. On the other hand, Miller-Rabin primality testing is a probabilistic primality test.

① Asymptotic complexity:

AKS determines certainty whether an integer is prime or composite. Its time complexity in bit-length n :

$$\text{poly}(\log n)$$

But Miller-Rabin uses randomly selected base to calculate whether n is composite or not. Due to modular exponentiation, the complexity:

$$O((\log n)^3)$$

for t random rounds, complexity:

$$O(t(\log n)^3)$$

2. Error probability:

Since AKS is deterministic; hence

$$p(\text{error}) = 0$$

The error probability of Miller-Rabin reduces with its round number:

$$p(\text{false prime}) \leq \frac{1}{4}$$

for t rounds.

$$p(\text{error}) \leq \left(\frac{1}{4}\right)^t$$

3. Practical Relevance to RSA:

For 2048-bit and 4096-bit RSA, prime generation requires testing large ~~calculation~~ candidate integers.

AKS is theoretically applicable but is generally not the practical choice because of its computational cost.

Miller-Rabin can test many candidates efficiently and repeated rounds make the error probability extremely small.

Question 6: Derive the extended euclidean algorithm formally (as a sequence of linear combinations), then use it to compute $\text{gcd}(240, 46)$ together with the Bezout coefficients x, y such that $240x + 46y = \text{gcd}(240, 46)$

$\Rightarrow$ The extended euclidean algorithm is used to find the gcd of two integers a and b , along with integers x and y satisfying:

$$ax + by = \text{gcd}(a, b)$$

Let, $r_0 = a, r_1 = b$

by euclidean division,

$$r_{i-1} = q_i r_i + r_{i+1}$$

$$\text{or, } r_{i+1} = r_{i-1} - q_i r_i$$

$$\text{initially, } r_0 = 1a + 0b$$

$$r_1 = 0a + 1b$$

$$\text{Let, } r_{i-1} = s_{i-1}a + t_{i-1}b$$

$$\text{and } r_i = s_i a + t_i b$$

then,

$$r_{i+1} = r_{i-1} - q_i r_i$$

$$\begin{aligned} &= (s_{i-1}a + t_{i-1}b) - q_i(s_ia + t_ib) \\ &= (s_{i-1} - q_i s_i)a + (t_{i-1} - q_i t_i)b \end{aligned}$$

Hence,

$$s_{i+1} = s_{i-1} - q_i s_i$$

$$\text{and, } t_{i+1} = t_{i-1} - q_i t_i$$

GCD(240, 46) using extended euclidean:

$$s = s_1 - s_2 \times q$$

$$t = t_1 - t_2 \times q$$

q	r ₁	r ₂	r	s ₁	s ₂	s	t ₁	t ₂	t
5	240	46	10	1	0	1	0	1	-5
4	46	10	6	0	1	-4	1	-5	21
1	10	6	4	1	-4	5	-5	21	-26
1	6	4	2	-4	5	-9	21	-26	47
2	4	2	0	5	-9	23	-26	47	-120

$$\text{GCD} = 2$$

Bezout coefficients; $s = -9, t = 47$

Question 7: state and prove the Chinese Remainder Theorem for pairwise coprime modulo. Apply it to solve $x \equiv 2 \pmod{3}$, $x \equiv 1 \pmod{5}$, $x \equiv 4 \pmod{7}$, $x \equiv 5 \pmod{11}$

$\Rightarrow$ Theorem:

Let $m_1, m_2, \dots, m_n$ be pairwise positive coprime integers.

$$\gcd(m_i, m_j) = 1 \quad ; i \neq j$$

then the system for integers $a_1, a_2, \dots, a_n$:

$$x \equiv a_i \pmod{m_i}$$

has a solution of unique modulo:

$$M = m_1 m_2 \dots m_n.$$

Proof of existence:

Let $M = m_1 m_2 \dots m_n$

$$\text{and } M_i = \frac{M}{m_i}$$

Since they are pairwise coprime, $\gcd(M_i, m_i) = 1$.

Therefore, M_i has the inverse modulo m_i . Let the inverse be y_i . so,

$$M_i y_i \equiv 1 \pmod{m_i}$$

Now,

$$x = a_1 M_1 y_1 + \dots + a_n M_n y_n$$

when we take this equation modulo m_i , all other terms become 0.

$$a_i M_i y_i \equiv a_i \pmod{m_i}$$

thus, $x \equiv a_i \pmod{m_i}$

Therefore solution exists.

Proof of uniqueness:

Let x and x' are two solutions, then;

$$x \equiv x' \pmod{m_i}$$

so, $m_i \mid (x - x')$, for every i .

Since all m_i 's are pairwise coprime,

$$m_1 m_2 \dots m_n \mid (x - x')$$

therefore, $x \equiv x' \pmod{M}$.

Hence the solution is unique modulo M .

Given,

$$x \equiv 2 \pmod{3}$$

$$x \equiv 3 \pmod{5}$$

$$x \equiv 4 \pmod{7}$$

$$x \equiv 5 \pmod{11}$$

$$M = 3 \times 5 \times 7 \times 11 = 1155$$

$$M_1 = \frac{1155}{3} = 385$$

$$M_2 = \frac{1155}{5} = 231$$

$$M_3 = \frac{1155}{7} = 165$$

$$M_4 = \frac{1155}{11} = 105$$

Finding the inverse, $M_i y_i \equiv 1 \pmod{m_i}$

For 3:

$$385 \equiv 1 \pmod{3}; y_1 = 1$$

For 5:

$$231 \equiv 1 \pmod{5}; y_2 = 1$$

For 7:

$$165 \equiv 4 \pmod{7}$$

$$4 \times 2 = 8 \equiv 1 \pmod{7}; y_3 = 2$$

For 11: $105 \equiv 6 \pmod{11}$

$$105 \equiv 6 \pmod{11}$$

$$ax \equiv 12 \equiv 1 \pmod{11}; y_4 = 2$$

Using CRT formula:

$$x = a_1 M_1 y_1 + a_2 M_2 y_2 + a_3 M_3 y_3 + a_4 M_4 y_4$$

$$= 2(385)1 + 3(231)1 + 4(165)2 + 5(105)2$$

$$= 3833$$

$$\cancel{3833} = 368$$

$$3833 - 3(1155) = 368$$

$$\therefore x \equiv 368 \pmod{1155}$$

$$\text{Hence, } x = 368 + 1155K; K \in \mathbb{Z}$$

Question 8: Devise the CRT-based RSA decryption / signing optimization from first principle and demonstrate the ~4x speed up with fully worked numerical example using small primes p, q .

⇒ CRT-based RSA optimization:

$$n, \text{ RSA}, n = pq$$

and decryption/signing normally computed as:

$$m = c^d \pmod{n}$$

Using CRT, this large computation is divided into two smaller computations:

~~$$d_p = d \pmod{p-1}$$~~

$$m_p = c^{d_p} \pmod{p}$$

$$m_q = c^{d_q} \pmod{q}$$

For CRT, d is separately taken in $p-1$ and $q-1$ modulo

$$d_p = d \pmod{p-1}$$

$$d_q = d \pmod{q-1}$$

To combine the result, compute:

$$q_{inv} = q^{-1} \pmod{p}$$

such that,

$$q(q_{inv}) \equiv 1 \pmod{p}$$

$$\text{Let, } h = q_{inv}(m_p - m_q) \pmod{p}$$

then the final result;

$$m = m_q + hq$$

$$\text{Let, } p = 11, q = 13$$

$$\text{then, } n = pq = 143$$

$$\text{Again, } \phi(n) = (11-1)(13-1) = 120$$

$$\text{Let, } e = 7$$

$$\text{then, } ed \equiv 1 \pmod{120}$$

$$\Rightarrow 7 \times 103 \equiv 1 \pmod{120}$$

$$\therefore d = 103$$

$$\text{Let ciphertext: } c = 4^2$$

$$\begin{aligned} d_p &= d \bmod (p-1) \\ &= 103 \bmod 10 \\ &= 3 \end{aligned}$$

$$\begin{aligned} d_q &= d \bmod (q-1) \\ &= 103 \bmod (12) \\ &= 7 \end{aligned}$$

$$\begin{aligned} m_q &= 42^7 \bmod 13 \\ &= 3^7 \bmod 13 \\ &= 3 \end{aligned}$$

$$\begin{aligned} m_p &= 42^3 \bmod 11 \\ &= 3 \end{aligned}$$

$$\begin{aligned} q_{inv} &= q^{-1} \bmod p \\ &= 13^{-1} \bmod 11 \\ &= 6 \end{aligned}$$

$$\begin{aligned} h &= q_{inv} (m_p - m_q) \bmod p \\ &= 6 (3 - 3) \bmod 11 \\ &= 0 \end{aligned}$$

$$m = m_q + hq = 3 + 0(13) = 3$$

$\therefore m = 3$, decrypted message.

Question 9: Explain the Bellare fault-injection attack on RSA-CRT signatures: how a single faulty modular computation during signing leaks a factor of n , and what countermeasures prevent it.

⇒ The Bellare attack is a fault attack against RSA implementations that use the CRT.

In RSA-CRT signing, the signer computes:

$$s_p = m^{d_p} \pmod{p}$$

$$s_q = m^{d_q} \pmod{q}$$

and combine them to find the correct signature s .

Attack process:

- (i) The attacker causes a fault in one CRT computation for example s_p .
- (ii) The device produces a faulty signature s' , while others are correct.
- (iii) The faulty signature satisfies the correct RSA relation modulo one prime prime but not other.
- (iv) Therefore $\gcd(s-s', n)$, reveals one secret factor p or q .

@ Once p is a sound,

$$q = \frac{n}{p}$$

Thus the attacker can factor $n = pq$ and potentially recover the RSA private key.

Countermeasures to prevent:

Verify the signature before releasing it:

$$s^e \bmod n = m$$

if verification fails, the signature is not released.

① Use redundant/repeated computation to detect faults.

② Use RSA binding to make fault injection harder.

Question 10: Compute the modular inverse using both the extended euclidean and Fermat's-Little theorem based exponentiation and discuss the trade-offs.

⇒ suppose, we need to find,

$$3^{-1} \pmod{7}$$

① Using extended euclidean (EEA)

Applying euclidean algorithm:

$$7 = 2 \times 3 + 1$$

$$\text{so, } 1 = 7 - 2(3)$$

comparing with, $1 = 7x + 3y$

$$\text{we get, } y = -2$$

$$\text{so, } 3^{-1} \equiv -2 \equiv 5 \pmod{7}, \quad 3 \times 5 = 15 \equiv 1 \pmod{7}$$

② Using Fermat's Little theorem:

For prime p ,

$$a^{p-1} \equiv 1 \pmod{p}$$

$$\text{therefore } a^{-1} \equiv a^{p-2} \pmod{p}$$

Here, $p = 7$, $a = 3$

$$3^{-1} \equiv 3^{7-2} = 3^5 \pmod{7}$$

$$3^5 \equiv 5 \pmod{7}$$

$$\therefore 3^{-1} \equiv 5 \pmod{7}$$

Trade offs for large modulus

① EEA: Usually faster and more efficient for computing a single inverse, it works only when $\gcd(a, m) = 1$.

② Fermat: Requires the modulus to be prime and uses modular exponentiation, which can be computationally heavy.

③ For large prime modulus, EEA is generally preferred for directly finding an inverse, while Fermat-based exponentiation is useful when fast modular exponentiation is already available.

Question: 11: Playfair cipher encryption and decryption of a plaintext minimum 16 letters with keyword of own choice.

⇒ Let, keyword = MONARCHY

Plaintext = CRYPTOGRAPHYTEST

Key Matrix:

M	O	N	A	R
C	H	Y	B	D
E	F	G	I/J	K
L	P	Q	S	T
U	V	W	X	Z

Diagram:

CR YP TO GR AP HY TE ST

Encryption: CR YP TO GR AP HY TE ST
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
 DM HQ PR KN OS YB LK TL

Ciphertext: DM HQ PR KN OS YB LK TL

Decryption: DM HQ PR KN OS YB LK TL
 ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
 CR YP TO GR AP HY TE ST

Plaintext: CR YP TO GR AP HY TE ST

Question 12: Critically analyze the structural weakness of the Playfair cipher against digraph frequency analysis and explain why it remains vulnerable to known-plaintext attack.

⇒ Playfair cipher is stronger than a monoalphabetic substitution because it encrypts pairs of letters rather than individual letters.

Yes it has important weaknesses.

① Vulnerability to Digraph Frequency analysis:

Playfair replaces each plaintext digraph with a ciphertext but rules are fixed. The attacker can guess the plaintext from frequent patterns.

② Known plaintext attack: Attackers can predict the plaintext from decryption algorithm when plaintext and corresponding ciphertext is known.

(ii) Stronger than monoalphabetic but still weak:

Monoalphabetic substitution preserves individual-letter frequency directly, but plaintext use digraph.

Still it has a small fixed 5×5 structure and deterministic encryption rules. Hence, sufficient plaintext and ciphertext can expose the rule.

Question 13: Derive and trace two full rounds of a simplified Feistel-structured cipher (DES-like) showing $L_0, R_0 \rightarrow L_1, R_1 \rightarrow L_2, R_2$ symbolically, then with the numeric values $L_0 = 0xAAAA AAAA$, $R_0 = 0x0F0F0F0F$, $K_1 = 0x0F0F0F0F$, $K_2 = 0x33333333$ using XOR as the round function.

$\Rightarrow$ Symbolic Feistel round equations:

$$L_1 = R_0 \quad R_1 = L_0 \oplus f(R_0, K_1) = L_0 \oplus R_0 \oplus K_1$$

$$L_2 = R_1 \quad R_2 = L_1 \oplus f(R_1, K_2) = L_1 \oplus R_1 \oplus K_2$$

Numeric trace

$$L_0 = 0xAAAAAAAA, R_0 = 0xF0F0F0F0$$

$$K_1 = 0x0F0F0F0F, R_1 = 0x33333333$$

Round 1:

$$L_1 = R_0 = 0xF0F0F0F0$$

$$f(R_0, K_1) = R_0 \oplus K_1 = 0xF0F0F0F0 \oplus 0x0F0F0F0F \\ = 0xFFFFFFFF$$

$$R_1 = L_0 \oplus f(R_0, K_1) = 0xAAAAAAAA \oplus 0xFFFFFFFF \\ = 0x55555555$$

Round 2:

$$L_2 = R_1 = 0x55555555$$

$$L(R_1, K_2) = R_1 \oplus K_2 = 0x66666666$$

$$R_2 = L_1 \oplus L(R_1, K_2) = 0xFFFFFFFF \oplus 0x66666666 \\ = 0x96969696$$

Final: $L_2 = 0x55555555, R_2 = 0x96969696$.

Question 14: Formally define the avalanche effect in block cipher. Using a toy 8-bit substitution-permutations network of your own construction, demonstrate how flipping a single input bit changes at least half the output bits.

⇒ The avalanche effect is the property of a good cryptographic algorithm whereby a small change in the input causes a large, unpredictable change in the output.

Toy 8-bit SPN construction:

Split 8-bit into two 4-bit nibbles → apply an s-box to each nibble → apply a bit permutation across all 8 positions.

$P = 10110010 \rightarrow L = 1011, R = 0010$

s-box outputs used: $S(1011) = 0100, S(0010) = 1101$

After substitution: 01001101

$c = 00011110$

$p' = 00110010$ bit 1 flipped

$\rightarrow L' = 0011, R = 0010 \quad S(0011) = 1011$

After substitution = 1011101

$c' = 10111011$

Why it matters: This confirms the avalanche criterion the s-box provides strong local diffusion and the permutation layer spreads changed bits across the whole block, without a strong avalanche effect, similar plaintext would produce similar ciphertext, letting attackers use pattern-matching.

Question 15: Evaluate the brute-force resistance of an 8-character alphanumeric password (62 alphabet) versus a 6-digit PIN, under a modern GPU cracking rate of 10^9 guesses/s.

$$\Rightarrow \text{GPU cracking rate} = 10^9 \text{ guesses/s}$$

Case 1: For 8-character alphanumeric password (62-character alphabet) total possible combinations:

$$N = 62^8 = 218,340,105,584,896 \approx 2.18 \times 10^{14}$$

worst case time:

$$T_{\text{worst}} = \frac{62^8}{10^9} \approx 218,340 \text{ seconds} \\ \approx 2.5 \text{ days.}$$

Expected case time:

$$T_{\text{expected}} = \frac{T_{\text{worst}}}{2} \\ \approx 1.26 \text{ days}$$

Case 2: 6 digit PIN (0-9)

$$N = 10^6 = 1,000,000$$

$$T_{\text{worst}} = \frac{10^6}{10^9} = 0.001 \text{ s} \\ = 1 \text{ millisecond}$$

$$T_{expected} = \frac{0.001}{2} = 0.0005 \text{ s} \\ = 0.5 \text{ milliseconds.}$$

Implication for Password Policy Design:

- (i) PINs are catastrophically weak
- (ii) Alphanumeric passwords are dramatically stronger.
- (iii) Policy: system should enforce minimum length and character.

Question 16: Compare the error propagation and parallelization characteristics of ECB, CBC, CFB, OFB and CTR modes using labelled diagrams for each. Recommend the most suitable mode for (a) encrypting a live video stream over a lossy network and (b) encrypting a static disk image.

⇒ ECB: ECB encrypts each block independently. It reveals patterns because identical plaintext blocks always produce identical ciphertext blocks.

CBC: XORs each plaintext with the previous ciphertext block before encryption.

CFB: Behaves similarly to CBC for error propagation.

OFB: Generates a keystream independently of the ciphertext.

CTR: Works like a stream cipher - Keystream = encrypt.

Recommendation:

(a) Live video stream over a lossy network: → CTR mode

Here CTR is ideal because:

(i) It's fully parallelizable.

(ii) A single bit error only affects that one bit.

(iii) Each block only needs its counter value.

(b) static disk image $\rightarrow$ CBC or XTS-like mode.

CBC is preferred over ECB because it hides repeating plaintext over ~~ECB~~ patterns, since decryption in CBC can still be parallelized.

Question 17: Using partial AES S-box below, perform the subbytes transformation on the state word $[0x23, 0xA7, 0x4C, 0x19]$, then apply ShiftRows to a 4×4 state matrix.

$\Rightarrow$ subbytes using S-box (row = high nibble, column = low nibble):

Input byte	High nibble	Low nibble	S-box output
0x23	2	3	D4
0xA7	A	7	63
0x4C	4	C	2E
0x19	1	9	C6

After subBytes: [D4, 63, 2E, C6]

step 2: Build 4x4 state matrix:

$$\begin{bmatrix} D4 & 63 & 2E & C6 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

step 3: shift rows:

Row 0 (D4, 63, 2E, C6) : unchanged.

Rows 1, 2, 3: all zeros $\rightarrow$ shifting now visible else

Final matrix:

$$\begin{bmatrix} D4 & 63 & 2E & C6 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \\ 00 & 00 & 00 & 00 \end{bmatrix}$$

Question 18: Derive the AES-128 Rijndael key schedule and compute the first two round keys (k_1, k_2) from given 128-bit cipher key of your choosing.

→ Chosen 128-bit cipher key: 2B 7E 15 16 28 AE

D2 A6 AB F7 15 88 09 CF 4F 3C

initial words: ($w_0 - w_3$)

$$w_0 = 2B7E1516$$

$$w_1 = 28AED2A6$$

$$w_2 = ABF71588$$

$$w_3 = 09CF4F3C$$

computing $k_1 (w_4 - w_7)$

$g(w_3)$ function = subword (RotWord (w_3) $\oplus$ Rcon(1))

Rotword (w_3): 09CF4F3C $\rightarrow$ CF4F3C09 (left circular shift)

subword: $S(CF) = 8A$, $S(4F) = 84$, $S(3C) = EB$,
 $S(09) = 01 \rightarrow 8A84EB01$

$$Rcon(1) = 01000000$$

$$g(w_3) = 8A84EB01 \oplus 01000000 = 8B84EB01$$

$$w_4 = w_0 \oplus g(w_3) = 2B7E1516 \oplus 2B74EB01 \\ = 10FAFE17$$

$$w_5 = w_4 \oplus w_1 = 88542CB1$$

$$w_6 = w_5 \oplus w_2 = 23A33939$$

$$w_7 = w_6 \oplus w_3 = 2A6C7605$$

$$k_1 = 10FAFE17 \quad 88542CB1 \quad 23A33939 \quad 2A6C7605$$

Question 19: Explain the mixcolumn transformation as polynomial multiplication over $GF(2^8)$ modulo $x^8 + x^4 + x^3 + x + 1$ works ~~together~~ through the full multiplication of one input column by the fixed Mixcolumns matrix.

$\Rightarrow x^8 + x^4 + x^3 + x + 1$ (0x11B), AES irreducible polynomial.

Multiplying by 02 means shift left by 1 bit, if the original MSB was 1, XOR the result with 0x1B. Multiplying by 03 = x -time (byte) $\oplus$ byte (since $03 = 02 \oplus 01$).

Matrix:

$$\begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}$$

input column: $[D4, BF, 5D, 30]$

Byte 0: 2. $D4 \oplus$ 3. $BF \oplus$ 1. $5D \oplus$ 1. 30

2. $D4$: $D4 = 11010100 \rightarrow \text{shift left} = 10101000$

MSB was 1 $\rightarrow$ XOR 0x1B $\rightarrow B3$

3. BF : $\text{rotate}(BF)$: $BF = 10111111 \rightarrow 01111110$

MSB was 1 $\rightarrow$ XOR 1B $\rightarrow 65$, then $65 \oplus BF = DA$

sum: $B3 \oplus DA \oplus 5D \oplus 30 = 04$.

Byte 1: 1. $D4 \oplus$ 2. $BF \oplus$ 3. $5D \oplus$ 1. 30

2. $BF = 65$

3. $5D = 10111010$

$\therefore BA \oplus 5D = E7$

sum: $D4 \oplus 65 \oplus E7 \oplus 30 = 66$

Byte 2: 1. D4 $\oplus$ 1 BF $\oplus$ 2. 30 $\oplus$ 0. 30

$$1. 5D = 0A$$

$$3. 30 = 0110000000$$

$$= 60 \oplus 30$$

$$= 50$$

$$\text{Sum: } D4 \oplus BF \oplus 0A \oplus 50 = 81$$

Byte 3: 3. D4 $\oplus$ 1 BF $\oplus$ 2. 5D $\oplus$ 2. 30

$$3. D4 = 67$$

$$2. 30 = 60$$

$$\text{Sum} = 67 \oplus 0F \oplus 5D \oplus 60 = E5$$

output code: [04, 66, 81, E5]

Question 20: Critically evaluate AES' resistance to side-channel attacks, specially timing analysis and differential power analysis.

⇒ AES is mathematically secure against conventional cryptanalysis, but its implementation can leak secret information through side channels.

① Timing and cache-Timing Attacks: In a table based

AES implementation, the s-box is accessed using secret-dependent indices.

Countermeasures - constant: Use an s-box implementation whose execution time and memory access pattern do not depend on secret data.

② Differential Power Analysis: DPA observes the device's power consumption during AES operation.

Countermeasures → Masking: Randomly mask sensitive intermediate values, for example:

$$x' = x \oplus r.$$

Question 21: Derive the RSA key generation, encryption and decryption procedures from Euler's theorem and prove correctness of decryption.

$\Rightarrow$ RSA is based on the difficulty of factoring a large composite number.

① RSA key generation: Two primes p and q .

$$n = pq$$

$$\phi(n) = (p-1)(q-1)$$

Let, e be a public exponent:

$$1 < e < \phi(n), \text{gcd}(e, \phi(n)) = 1$$

choose an odd number d ;

$$ed \equiv 1 \pmod{\phi(n)}$$

public key: (e, n)

private key: (d, n)

(i) Encryption: For message M ,

$$c = M^e \pmod{n}$$

(ii) Decryption: The receiver calculates:

$$m' = c^d \pmod{n}$$

$$\therefore m' = M^{ed} \pmod{n}$$

(iii) Correctness of decryption:

$$\gcd(M, N) = 1$$

$$\text{since, } ed \equiv 1 \pmod{\phi(n)}$$

$$\text{hence, } ed = 1 + k \phi(n)$$

By Euler's theorem,

$$M^{\phi(n)} \equiv 1 \pmod{n}$$

$$\therefore M^{ed} = M^{1 + k \phi(n)}$$

$$= M (M^{\phi(n)})^k$$

$$\equiv M (1)^k$$

$$\therefore M^{ed} \equiv M \pmod{n}$$

Question 22: Given primes $p=11$, $q=13$, generate a full RSA keypair then sign a message hash $H(M)=23$ and verify the signature, showing all intermediate modular exponentiation steps.

$\Rightarrow p=11, q=13$

① RSA key pair:

$$n = pq = 143$$

$$\phi(n) = 10 \times 12 = 120$$

choose $e=7$

$$\text{since, } 7(103) = 721 = 6(120) + 1$$

$$\therefore d = 103$$

$\therefore$ Public key = $(7, 143)$

private key = $(103, 143)$

2. Hash signing:

$$H(M) = 23$$

RSA signature:

$$s = H(M)^d \mod n$$

$$s = 23^{103} \mod 143$$

$$103 = (1100111)_2 = 64 + 32 + 4 + 2 + 1$$

sequential squaring:

$$23^1 \mod 143 = 23$$

$$23^2 \mod 143 = 100$$

$$23^4 \mod 143 = 133$$

$$23^8 \mod 143 = 100$$

$$23^{16} \mod 143 = 133$$

$$23^{32} \mod 143 = 100$$

$$23^{64} \mod 143 = 133$$

$$\therefore 23^{103} \equiv 133 \times 100 \times 133 \times 100 \times 133 \pmod{143}$$

we get, $133 \times 100 \equiv 1 \pmod{143}$

$$\therefore s = 23$$

③ Verifying signature:

$$H' = s^e \bmod n$$

$$= 23^7 \bmod 143$$

$$23^2 \equiv 100 \bmod 143$$

$$23^4 \equiv 133 \bmod 143$$

$$\therefore 23^7 = 23^4 \times 23^2 \times 23$$

$$\equiv 133 \times 100 \times 23 \bmod 143$$

$$\equiv 1 \times 23$$

$$\therefore H' = 23$$

$\therefore$ the signature is valid.

Question 23: Derive the general formula for IPsec ESP overhead percentage, in tunnel mode as a function of payload size, header size, padding.

⇒ In IPsec ESP tunnel mode, the original IP packet is encapsulated inside a new ESP-protected packet.

Let, p = payload size

H = ESP header size

A = Authentication data size

L = Padding size

total ESP overhead:

$$O = H + A + L$$

overhead relative to payload:

$$\text{Overhead \%} = \frac{H + A + L}{p} \times 100 \%$$

Given, $P = 1200$ bytes

$H = 8$ "

$A = 16$ "

$L = 2$ "

$$\text{overhead \%} = \frac{8 + 16 + 2}{1200} \times 100\%$$
$$= 2.17\%$$

Therefore the protected packet carries 26 bytes of specified ESP overhead for a 1200 byte payload.

Question 24: Execute a full diffie-hellman key exchange (Aleya) with $p = 23$, $g = 5$, private keys $a = 6$, and $b = 15$ (Badol) confirming both parties derive the same shared secret.

$\Rightarrow$ Given, $p = 23$, $g = 5$

Private keys: $a = 6$ (Aleya)
 $b = 15$ (Badol)

⑩ Alaya's public key:

$$A = g^a \pmod{p}$$
$$= 5^6 \pmod{23}$$

$$A = 5^6 \pmod{23}$$
$$= 5^4 \times 5^2 \pmod{23}$$
$$\equiv 17 \pmod{23}$$

$$A = 17$$

⑪ Badol's public key:

$$B = 5^{15} \pmod{23}$$
$$5^{15} = 5^8 \times 5^4 \times 5^2 \times 5$$
$$\equiv 18 \pmod{23}$$

$$\therefore B = 18$$

⑫ Shared secret:

From Alaya:

$$K = 18^6 \pmod{23}$$

$$18^6 \equiv 4 \times 2 = 8$$

$$\therefore K = 8$$

From Dadol,

$$R = 17^{15} \pmod{23}$$

$$17^{15} = 17^8 \cdot 17^4 \cdot 17^2 \cdot 17$$

$$\therefore K = 8$$

Both parties share the same secret key.

Question 25: Compare the computational cost of establishing a shared key via RSA v/s Diffie-Hellman key agreement at an equivalent security level.



Feature	RSA Key Transport	ECDH Key Agreement
Main operation	RSA encryption + private key decryption	Key agreement using exponentiation multiplication
Key size	Longer	Much smaller
Computation	RSA public operation can be fast, but private operation is expensive	ECDH scalar multiplication is efficient.
Communication	Longer public key	Smaller key and exchange values.

Forward secrecy: In RSA key transport, the server's long term private key R is used to recover the transported secret.

In ECDH, fresh temporary private keys are generated for each session.

Question 26: In an ElGamal scheme with $p=467$, $g=2$, private key $x=127$, message hash $H(M)=100$ and ~~message~~ $k=213$ (with $\gcd(k, p-1)=1$):

Compute the public key, generate the signature, and verify it.

$\Rightarrow$ Given, $p=467$, $g=2$, $x=127$, $H(M)=100$, $k=213$

$$p-1=466$$

Public key: $y = g^x \pmod{p}$

$$= 2^{127} \pmod{467}$$
$$= 132$$

$$\text{public key} = (p, g, y) = (467, 2, 132)$$

computing r :

$$\begin{aligned} r &= g^k \bmod p \\ &= 2^{213} \bmod 467 \\ &= 29 \end{aligned}$$

computing s :

$$s = k^{-1} (H(m) - x \cdot r) \bmod (p-1) :$$

$$\gcd(213, 466) = 1 \quad (\therefore k^{-1} \text{ exists})$$

$$k^{-1} = 213^{-1} \bmod 466 = 431$$

$$(213 \times 431 = 91803 = 197 \times 466 + 1)$$

$$x \cdot r = 127 \times 29 = 3683 \equiv 421 \pmod{466}$$

$$H - x \cdot r = 100 - 421 = -321 \equiv 145 \pmod{466}$$

$$\begin{aligned} s &= 431 \times 145 \bmod 466 \equiv 62495 \pmod{466} \\ &\equiv 51 \pmod{466} \end{aligned}$$

$$\text{signature}(r, s) = (29, 51)$$

Verification: check $g^{r^1} \cdot H(m) \equiv y^{r^1} \cdot r^{s^1} \pmod{p}$

$$2^{100} \bmod 467 = 189$$

$$y^{r^1} = 132^{29} \pmod{467} = 216$$

$$r^{s^1} = 216^{51} \pmod{467} = 176$$

$$\therefore 216 \times 176 \pmod{467} = 32016 \pmod{467} \\ = 1813$$

$\therefore \text{LHS} = \text{RHS}$. (valid signature)

Question 27: Prove algebraically that the ElGamal signature verification equation $g^H(M) \equiv y^n r^s \pmod{p}$ holds for correctly generated signature, starting from the definition $s \equiv k^{-1}(H(M) - xn) \pmod{p-1}$

$\Rightarrow$ From the signing equation:

$$s \equiv k^{-1}(H(M) - xn) \pmod{p-1}$$

multiplying both sides by k :

$$ks \equiv H(M) - xn \pmod{p-1}$$

Rearranging:

$$H(M) \equiv xn + ks \pmod{p-1}$$

Raise g to both sides, by Fermat's Little theorem, since $g \in \mathbb{Z}_p^*$, we have $g^{p-1} \equiv 1 \pmod{p}$. This gives the exact equality mod p :

$$g^{H(M)} \equiv g^{xn+ks} \pmod{p}$$

splitting the exponent:

$$g^{H(M)} \equiv g^{xn} \cdot g^{ks} \equiv (g^n)^n \cdot (g^k)^s \pmod{p}$$

substituting the definitions $y = g^n \pmod{p}$ (public key) and $n = g^k \pmod{p}$ (first signature component)

$$g^{H(M)} \equiv y^n n^s \pmod{p}$$

Question 28: On the elliptic curve $y^2 = x^3 + x + 1$ (mod 23), verify that $P(3, 10)$ lies on the curve, compute $2P$ via point doubling, and compute $P+Q$ for $Q(9, 7)$ showing the full modular arithmetic derivation.

⇒ Part 1: Verify $P(3, 10)$ lies on the curve.

$$\text{LHS: } y^2 = 10^2 = 100 \equiv 100 - 92 = 8 \pmod{23}$$

$$\text{RHS: } x^3 + x + 1 = 27 + 3 + 1 = 31 \equiv 8 \pmod{23}$$

LHS = RHS $\rightarrow P$ lies on the curve.

Part 2: Point doubling: $2P$

slope for doubling: $\lambda = (3x_1^2 + a) / 2y_1 \pmod{p}$ with

$$\text{Numerator: } 3(3^2) + 1 = 27 + 1 = 28 \equiv 5 \pmod{23}$$

$$\text{Denominator: } 2(10) = 20$$

$$20^{-1} \pmod{23} = 15 \text{ since } 20 \times 15 = 300 = (13 \times 23 + 1)$$

$$\lambda = 5 \times 15 = 75 \equiv 6 \pmod{23}$$

coordinates :

$$x_3 = 1^2 - 2x_1 = 36 - 6 = 30 \equiv 7 \pmod{23}$$

$$y_3 = 1(x_1 - x_3) - y_1 = 6(3 - 7) = 10 = -34 \equiv 12 \pmod{23}$$

$$2P = (7, 12) \text{ verify } y: 12^2 = 144 \equiv 6;$$

$$7^3 + 7 + 1 = 351 \equiv 6$$

Question 29: On the curve $y^2 \equiv x^3 + 7x + 10 \pmod{37}$ with

base point $G = (2, 5)$ of order $n = 19$ and private key $d = 9$; compute the public key $Q = dG$, generate an ECDSA signature (r, s) for $H(M) = 8$ using $k = 3$

$\Rightarrow$ substituting $G = (2, 5)$ into $y^2 \equiv x^3 + 7x + 10 \pmod{37}$

gives LHS = 25 but RHS = $8 + 14 + 10 = 32$, so, G does

not satisfy the stated curve. The constant is taken

as $b = 3$ for which $25 \equiv 8 + 14 + 3 = 25$ and $n = 19$ is

taken as the stated order of G as given.

Computing public key: $Q = dG = 9G$

$$2G: \lambda = (3 \cdot 4 + 7) / 2 \cdot 3 = 19/10; 10^{-1} = 26 \rightarrow \lambda = 13$$

$$x_3 = 162 - 4 = 17, y_3 = 13(2 - 17) - 9 = -200 \equiv 22 \rightarrow 2G = (17, 22)$$

$$4G = 2(2G): \lambda = (3 \cdot 17^2 + 7) / (44)$$

$$= 23/7; 7^{-1}$$

$$= 16 \rightarrow \lambda = 35$$

$$x_3 = 1225 - 3 \cdot 4 \equiv 7, y_3 = 35(17 - 7) - 22$$

$$= 328$$

$$\equiv 32 \rightarrow 4G = (7, 32)$$

$$8G = 2(4G): \lambda = 6/27; 27^{-1} = 11 \rightarrow \lambda = 29$$

$$x_3 = 341 - 14 \equiv 13; y_3 = -206 \equiv 16 \rightarrow 8G = (13, 16)$$

$$9G = 8G + G; \lambda = (5 - 16) / (2 - 13)$$

$$= (-11) / (-11) = 1$$

$$x_3 = 1 - 13 \cdot 1 - 2 = -14 \equiv 23, y_3 = 1(13 - 23) - 16 = -16 \equiv 11$$

$$Q(23, 11) - \text{verify: } 11^2 = 121 \equiv 10;$$

$$23^2 + 7(23) + 3 \equiv 10$$

Computing $3G = 3G$ (with $h = 3$): $3G = 2G + G$

$$\lambda = (5 - 22) / (2 - 17) = 17 / 15; 15 \cdot 1 = 5 \rightarrow \lambda = 81 \equiv 11$$

$$x_3 = 129 - 17 - 2 = 102 \equiv 28, y_3 = 11(17 - 22) - 22 \\ = -143 \equiv 5$$

$$\therefore 3G = (28, 5)$$

Computing n : $n = x\text{-coordinate mod } n = 28 \text{ mod } 19 = 9$

Computing s : $s = k^{-1}(H(m) + dn) \text{ mod } n$:

$$dn = 9 \times 9 = 81; H + dn = 8 + 81 = 89 \equiv 13 \pmod{19}$$

$$k^{-1} \equiv 3^{-1} \pmod{19} = 13 \quad (3 \times 13 = 39 \equiv 1)$$

$$s = 13 \times 13 = 169 \pmod{19} = 17$$

signature: $(n, s) = (9, 17)$

Question 30: Compare the key size security equivalence between RSA, ELGAMAL and ECC, and explain why ECC is preferred for resource-constrained devices.

⇒

Security	Symmetric	RSA / ELGAMAL key	ECC key
80 bit	80	1024 bits	160 bits
112 bit	112	2048 bits	224 bits
128 bit	128	3072 bits	256 bits
192 bit	192	7680 bits	384 bits
256 bit	256	15360 bits	521 bits

ECC is preferred for resource-constrained devices because:

- (i) Faster key generation
- (ii) Smaller key and certificates
- (iii) Less computation and power
- (iv) Lower bandwidth
- (v) Less RAM required.

Question 31: Formally define preimage resistance, second preimage resistance and collision resistance for a cryptographic hash function. Derive the birthday attack complexity for finding N -collision in an n -bit hash.

$\Rightarrow$ Let $H = \{0,1\}^* \rightarrow \{0,1\}^n$ be a cryptographic hash function.

(i) Preimage resistance (one-wayness): Given a hash output y , it must be computationally infeasible to find any message m such that $H(m) = y$. ~~that~~

(ii) Second preimage resistance: Given a specific message m , it must be infeasible to find a different message $m_2 \neq m$, with $H(m_2) = H(m)$.

(iii) Collision resistance: It must be infeasible to find any pair $m_1 \neq m_2$ such that $H(m_1) = H(m_2)$. Here the attacker is free to choose both messages.

Birthday Attack complexity: Suppose the attacker hashes q random messages. With $N = 2^n$ possible hash values, number of value pairs:

$$\binom{q}{2} = \frac{q(q-1)}{2} \approx \frac{q^2}{2}$$

Expected number of collisions:

$$E \approx \frac{q^2}{2N}$$

setting $E = 1$

$$\frac{q^2}{2N} \approx 1 \Rightarrow q \approx \sqrt{2N} = \sqrt{2 \cdot 2^n} \approx 2^{n/2}$$

$2^{1(n/2)}$ hash evaluations are needed.

Why 256-bit output is needed!

Hash size preimage cost collision cost

128 bit

$$2^{128}$$

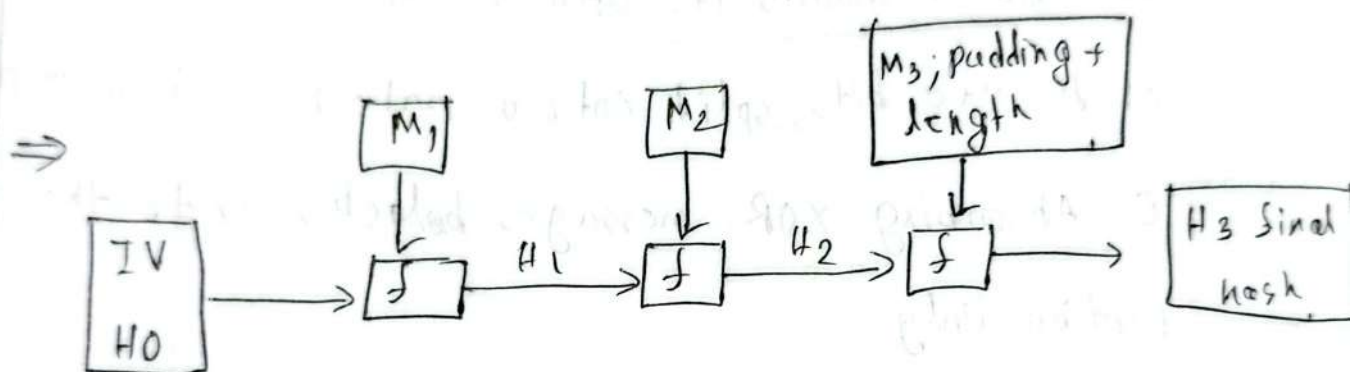
2^{64} x feasible total

256 bit

$$2^{256}$$

2^{128} secured.

Question 32: Trace the Merkle-Damgård construction with a diagram showing message blocks, compression functions calls. Explain length-extension attack that this construction is vulnerable to and how HMAC and SHA-3 avoid it.



The length-extension attack:

The published hash output is the entire internal state. So if an attacker knows $H(M)$ and the length of M , they can set $H_0' = H(M)$ and continue hashing additional blocks, computing H without even knowing M itself.

How HMAC avoids it: HMAC uses nested hashing with two derived keys:

$$\text{HMAC}(K, M) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel M))$$

The inner hash output is rehashed with the outer key.

How SHA-3 avoids it: SHA-3 keeps an internal state

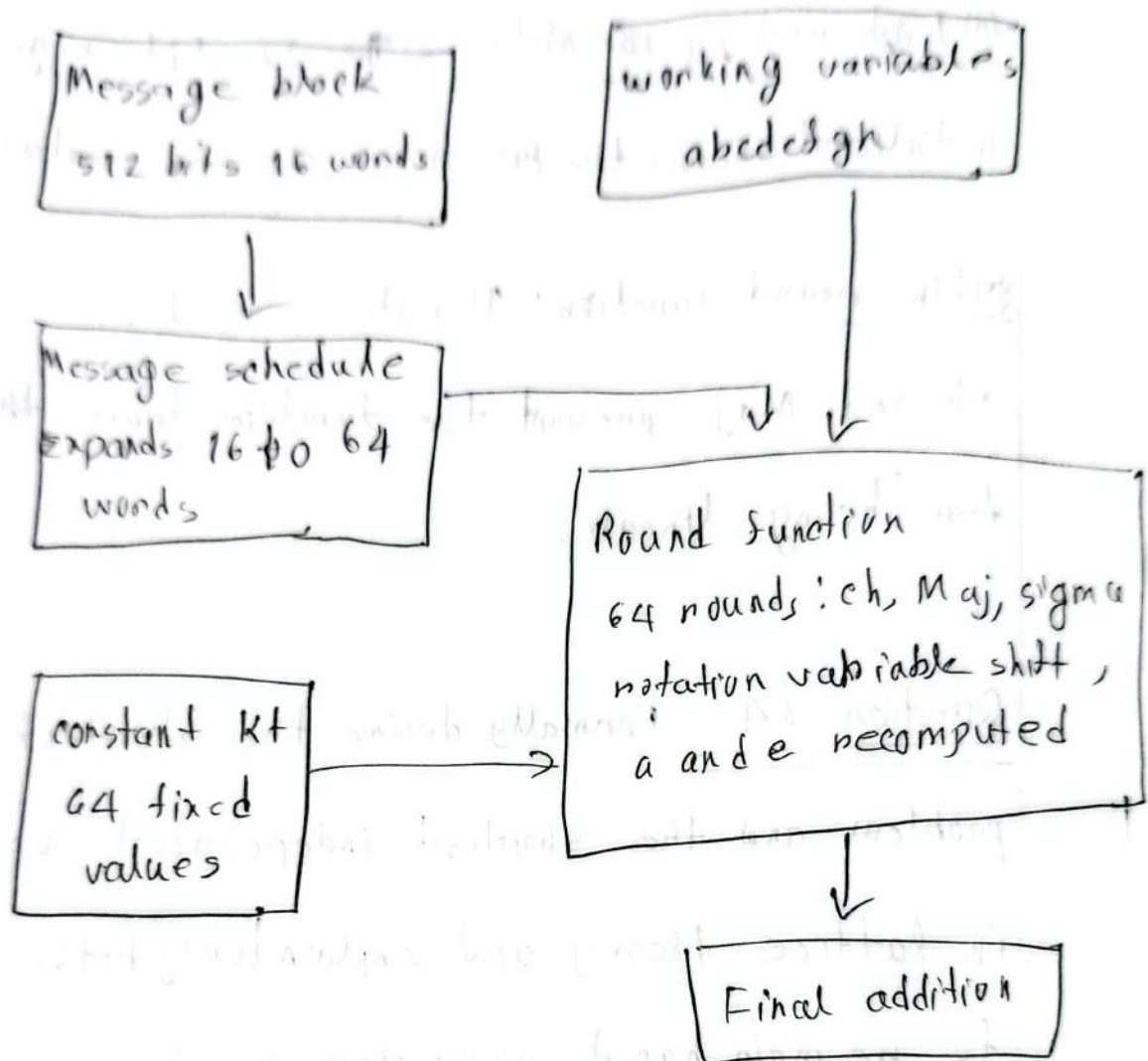
of $b = n + c$ bits, split into a rate r and a capacity

c . Absorbing XOR messages blocks into the rate portion only.

Question 33: Explain the High level internal structure

of the SHA-256 compression function—message schedule expansion, the eight working variables and the round function using a labelled block diagram.

⇒ The block diagram of SHA-256 compression:



① Message schedule expansion: The 512 bit input blocks is first split into sixteen 32-bit words. These are then expanded to 64 words, using the recurrence.

(v) Eight working variables: The 32-bit registers are initialized from the previous chaining value $H_0 \dots H_7$.

(vi) The round function: It role is confusion and nonlinearity and Maj prevent the function from ~~the beginning~~ being linear.

Question 34: Formally define the shortest vector problem and the shortest independent vectors problem in lattice theory and explain why both are believed to remain hard even for quantum computers.

⇒ Lattice: A discrete set of points $L(B) = \{ \sum z_i b_i : z_i \in \mathbb{Z} \}$ generated by integer combinations of basis vectors $b_1 \dots b_n$.

shortest vector problem: Given a Lattice basis, find the non-zero vector $v \in L$ with the smallest length. where, $\lambda_1(L)$ is the length of the shortest non-zero vector in the lattice.

shortest independent vector problem: Given a basis of an n -dimensional lattice, find n linearly independent vectors $v_1, \dots, v_n \in \mathbb{Z}$ such that the maximum length among them is at most $\lambda_n(L)$.

Why both resist quantum attack:

→ Shor's algorithm only works on abelian Hidden subgroup problem - this is why it breaks factoring and discrete logs. srp/SIVP instead reduce to the non-abelian dihedral HSP, for which no efficient quantum algorithm is known.

Question 35: Explain the core idea behind crystals

kyber's Modulo-Learning with errors key encapsulation mechanism, including how 'noise' is used to hide the secret.

⇒ Kyber's security is based on Modulo-LWE. A public matrix A and secret s produce $t = As + e \pmod{q}$ where e is a small random 'noise' vector.

How noise hides the secret: Since e is the small but random, t looks statistically indistinguishable from a uniformly random vector.

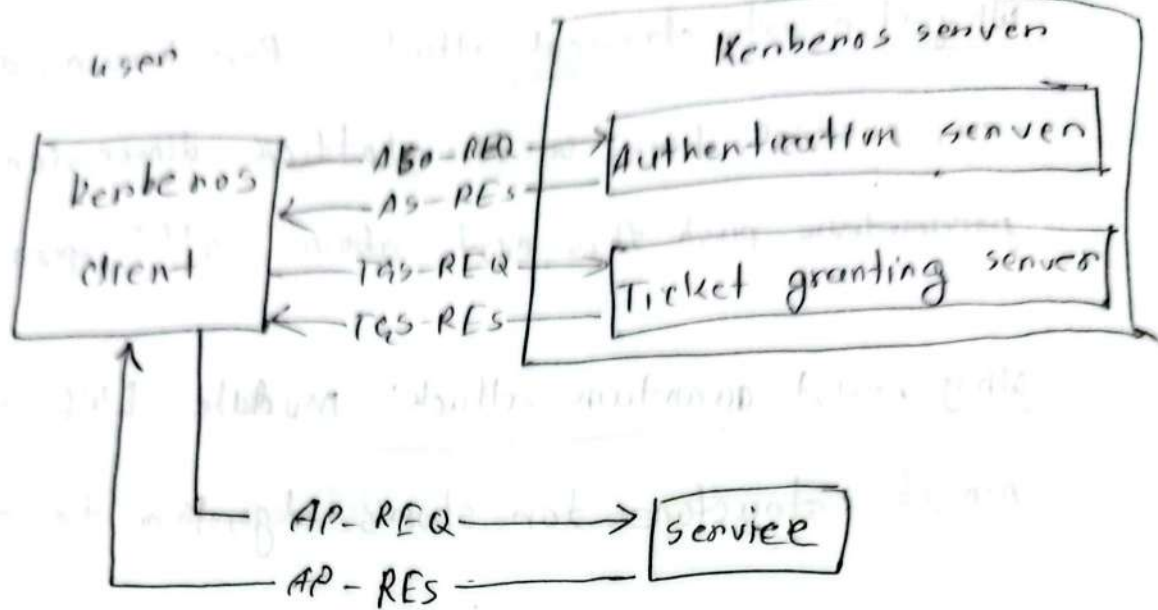
KEM flow: public key = sender picks small random r, e_1, e_2 ; sends ciphertext $(u = A^T r + e_1, v = A^T r + e_2 + \text{Encode}(m))$.

Why it resists classical attacks: Best known attacks run in exponential time in the lattice dimension; Kyber's parameters push this cost above 2^{128} operations.

Why resist quantum attack: Module LWE has no abelian periodic structure for Shor's algorithm to exploit.

Question 36: Diagram and explain the full Kerberos authentication protocol, covering the Authentication server (AS) exchange, ticket granting, server-exchange and client-server exchange.

⇒ Kerberos uses a trusted authentication server and ticket granting server so the user's password is not repeatedly sent.



① AS exchange: client sends its identity ID_C and requests a TGI.

$$\{K_C, tgs, TGI\}_{K_C}$$

where, $TGI = \{ID_C, lifetime, K_C, tgs\}_{K_{TGS}}$

② TGS exchange: client sends TGI and Authentication

$$Authentication = \{ID_C, timestamp\}_{K_C}, tgs$$

TGS verifies and creates a new session key K_C, s
 $\{K_C, s, lifetime\}_{K_C, tgs} + Tickets$

③ Client-server exchange: Client sends tickets and a new authentication to the service server.

The server decrypts the ticket and verifies the ~~auth~~ authentication.

TAT avoids repeated password transmission:

The password is never sent to the network. It is used to obtain k_c and decrypt the AS response. Then client uses the TAT and temporary session keys to obtain service ticket.

Question 37: Explain why a plain hash based MAC is vulnerable to a length extension attack and derive how HMAC's nested construction H eliminates this.

⇒ Why $H(K||M)$ is vulnerable: In Merkle-Damgård hashes, the output is the full internal state so given a valid pair $(M, T = H(K||M))$, an attacker can resume hashing from T and append any suffix M' , producing a valid tag for $K||M||\text{padding}||M'$ without even knowing K .

How HMAC fixes it: In HMAC the inner hash result is never published directly - it is fed into a second, outer hash keys with a different derived key to extend the tag, an attacker would need to compute the outer hash, which requires knowing $K \oplus \text{pad}$. Since, that's secret, extension attacks fail.

Using toy hash $f(h, m) = (h + m) \times 7 \bmod 256$, $IV = 5$, $k = 12$,
 $m = [30, 40]$,

HMAC: inner $J = H(15 || 30 || 40) = 162$ (using $k \oplus \text{ipad} = 15$),

outer $T = H(5 || 162) = 88$.

Extending 88 with 50 gives 108 but correct HMAC for $[30, 40, 50]$ requires rendering both inner and outer hashing with the secret key, giving different value.

Question 38: Diagram the TLS 1.3 handshake and explain how the session keys are derived via HKDF, ~~from~~ Compare TLS 1.2 and TLS 1.3.

⇒ Flowchart of TLS 1.3:

ClientHello → Server

Hello (+key-share) → [Everything after this point is encrypted]

Encrypted extensions. ←

Certificate, certificate verify, finished (server) → Finished (client)

Application Data ←

Key derivation: Both sides compute the (EC) DHE shared secret right after serverHello, then run it through HKDF-Extract / Expand chained with an early secret and handshake transcript hash to derive handshake traffic secrets and application traffic secrets.

Comparison:

	TLS 1.2	TLS 1.3
Round trips	2 RTT	1-RTT
Forward Secrecy	Optional (RSA key transport allowed)	Mandatory (DHE only)
Handshake encryption	Sent in plaintext	Encrypted after serverHello

Question 39: Design a bandwidth and duty cycle budget for an IOT sensor network in which each node transmits a 1200-byte packet once per minute over a LPWAN technology.

⇒ Data volume per node/day: $1200 \text{ bytes} \times 1440 \text{ (packets/day)}$
 $= 1728000 \text{ bytes} = 1687.5 \text{ kB/day}$.

Airtime issue (LoRaWAN, SF9): Max payload per frame ≈ 115 bytes, so 1200 bytes needs ~ 11 fragmented frames, each ~ 0.335 s airtime $\rightarrow \sim 3.63$ s per transmission $\times 60/\text{hour}$
 $\approx \sim 218$ s/hour of airtime.

Regulatory limit: EU - 1% duty cycle allows only 36 s/hour.

Required airtime vastly exceeds the allowed 36 s/hour - this design is non-compliant.

Fixes: compress payload (< 50 bytes), reduce transmission on frequency, use lower spreading factor or switch to

a licensed technology like NB-IOT/LTE-M with no duty-cycle cap.

Question 40: Critically assess a lightweight cryptographic standard suitable for resource constrained IOT devices against AES-128 in terms of memory footprint, energy consumption. Which is more appropriate for battery powered sensors.

⇒

Criterion	AESon-128	AES-128
Memory	~1-2 KB code, no lookup tables	Larger AES-ECM needs multiplication tables (256 B - 4 KB)
Energy/byte	Lower (simple bitwise permutation)	Higher (Mix columns + GHASH multiplication)
Side-channel resistance	Naturally constant-time (small bitsliced s-box), cheap to mask	Table-based lookups leak via cache-timing, masking is expensive.
Maturity	New	20+ years analysis.

Recommendation: For a battery powered sensor with <10KB RAM and no AES hardware acceleration, choose AES-128 - it combines AEAD+hashing in one small, table free, low-energy, naturally side-channel resistant primitive.